Massively Scalable GPU-Accelerated Framework for Dynamic Capacitated Vehicle Routing Problems

Senay Patel
CS22B061
Project Guide: Prof. Rupesh Nasre
Department of Computer Science and Engineering
Indian Institute of Technology Madras, Chennai, India

Abstract

While static Capacitated Vehicle Routing Problem (CVRP) metaheuristics have reached a high degree of maturity, exemplified by the Hybrid Genetic Search (HGS-CVRP) framework of Vidal [1], the construction of genuinely *dynamic* CVRP engines capable of instantaneous, localised route updates in response to real-time customer insertions and deletions remains comparatively under-explored. Existing approaches resort to expensive full-instance static re-optimisation after each perturbation, incurring latencies that are incompatible with modern real-time dispatch requirements. This paper bridges this gap by extracting and restructuring the high-performance inter-route and intra-route local search mechanics of HGS into an event-driven dynamic solver. The resulting `Inserter`/`Deleter` architecture treats each customer arrival or cancellation as a localised perturbation event, structurally modifying only the affected route segments and then invoking a Variable Neighbourhood Descent (VND) local search, adapted from the nine move operators and SWAP* of HGS, as a targeted *healing mechanism* that re-optimises the perturbed routes without reprocessing the entire solution. To enable this dynamic framework to scale to massive, industrially relevant networks of 100,000+ nodes on commodity GPU hardware, we eliminate the $\mathcal{O}(N^2)$ dense distance matrix and best-insertion cache that underpin standard HGS, replacing them with on-the-fly Euclidean distance computation over Structure-of-Arrays (SoA) coordinate layouts and sparse, lazily computed insertion caches, reducing memory from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$. The granular k-nearest-neighbour construction is parallelised via custom CUDA kernels with per-thread register-resident partial selection, achieving sub-second neighbourhood computation for instances up to 100,000 nodes. Experimental results demonstrates that our dynamic framework processes single-node insertion and deletion events approximately $10\times$ faster than static re-solving, scaling to 500,000+ nodes on a single GPU workstation, a scale entirely out of reach for conventional dense-matrix solvers.

Keywords: Dynamic Vehicle Routing, Event-Driven Optimisation, CUDA, GPU Computing, Local Search, HGS-CVRP, Granular Neighbourhoods, Scalability.

1 Introduction

The Capacitated Vehicle Routing Problem (CVRP) asks for a minimum-cost set of routes originating and terminating at a central depot such that every customer is visited exactly once and no vehicle exceeds its carrying capacity [3]. As an $\mathcal{NP}$-hard generalisation of the Travelling Salesman Problem, CVRP has attracted sustained research interest spanning exact methods, metaheuristics, and, more recently, machine-learning-guided approaches.

1.1 The Dynamic Routing Gap

Classical CVRP formulations assume a fixed, fully known customer set. In practice, modern logistics environments, same-day delivery networks, ride-hailing platforms, and emergency response systems, demand *dynamic* routing capabilities: customers appear (e.g., new online orders) and disappear (e.g., cancellations) during the operational horizon. The *Dynamic Capacitated Vehicle Routing Problem*

Source code and benchmark instances available at: https://github.com/senay73629/Dynamic-CVRP

(D-CVRP) extends the static formulation by requiring that an existing feasible solution be updated in response to such perturbations, ideally within sub-second latency to support real-time dispatching.

Despite the practical importance of D-CVRP, the overwhelming majority of research effort has been directed at static formulations. The few dynamic approaches that exist [12] typically fall back on full-instance static re-optimisation after each customer event, a strategy that scales poorly and squanders the structural investment embedded in the existing solution. A true dynamic engine must instead perform *localised*, *sub-second* route surgery: modify only the affected region of the routing network and invoke a targeted local search to heal the perturbation, rather than reprocessing all N nodes from scratch.

1.2 The Computational Wall at Scale

Even if a dynamic routing architecture is designed, deploying it at industrially relevant scales faces a hard engineering barrier. State-of-the-art CVRP solvers, including HGS-CVRP of Vidal [1] and the open-source PyVRP [5] library, rely on $\mathcal{O}(N^2)$ pairwise distance matrices to enable constant-time distance lookups during local search. While effective for benchmark instances of up to $\sim$10,000 nodes, this architecture encounters a hard memory ceiling at larger scales. For $N = 100{,}000$, a double-precision distance matrix requires approximately $100{,}000^2 \times 8 \approx 74.5\,\text{GB}$ of RAM; at $N = 500{,}000$, the requirement exceeds 1.86 TB, far beyond the capacity of any commodity workstation. Even when memory is available, the $\mathcal{O}(N^2)$ initialisation cost dominates the solver runtime and precludes the sub-second response times essential for dynamic dispatch. A scalable dynamic framework therefore requires not only a novel event-driven architecture, but also a fundamental re-engineering of the underlying data structures.

1.3 Contributions

This work makes the following principal contributions, listed in order of conceptual significance:

- **Core Novelty , Event-Driven Dynamic CVRP Framework.** We design and implement a localised, sub-second dynamic insertion and deletion framework for CVRP via the unified `Inserter`/`Deleter` architecture. Rather than resorting to computationally expensive full-instance static re-optimisation after each customer event, our framework treats each insertion or deletion as a localised perturbation, structurally modifies only the affected route segments via linked-list surgery, and invokes a Variable Neighbourhood Descent (VND) local search, adapted from the nine move operators and SWAP* of HGS-CVRP, as a targeted *healing mechanism* to re-optimise the perturbed routes. This yields approximately 10× faster event processing compared to static re-solving across all tested scales.

- **Enabling Engineering , Elimination of $\mathcal{O}(N^2)$ Memory Bottlenecks.** To enable the dynamic framework to scale to massive, industrially relevant networks, we eliminate the two principal $\mathcal{O}(N^2)$ memory structures of standard HGS: the dense distance matrix is replaced by on-the-fly Euclidean distance computation over Structure-of-Arrays (SoA) coordinate layouts, and the SWAP* best-insertion cache is replaced by lazily computed $\mathcal{O}(N)$-sized sparse vectors. These substitutions reduce the memory footprint from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, enabling seamless scaling to 500,000+ nodes on a single GPU workstation.

- **HPC Acceleration , GPU-Parallelised Neighbourhood Construction.** We parallelise the granular k-nearest-neighbour pre-computation, the dominant preprocessing cost, via custom CUDA kernels and Thrust reductions. Each client node is mapped to an independent thread performing in-register partial k-selection, achieving sub-second neighbourhood computation for instances up to 100,000 nodes and maintaining interactive execution bounds essential for real-time dynamic dispatch.

The remainder of this paper is organised as follows. Section 2 reviews the relevant literature on HGS, granular local search, and GPU-accelerated vehicle routing. Section 3 presents the mathematical

formulation, the event-driven dynamic architecture, the HGS-adapted local search healing mechanism, and the memory-decoupling and CUDA parallelisation techniques that enable the framework to operate at scale. Section 4 describes the experimental framework and provides structured placeholders for empirical results. Section 5 concludes with a summary and future research directions.

2 Background and Related Work

2.1 The Hybrid Genetic Search Framework

The Hybrid Genetic Search (HGS) framework introduced by Vidal [2, 1] represents the current state of the art for static CVRP instances, consistently achieving near-optimal or best-known solutions across the standard CVRPLIB benchmarks. HGS combines a genetic algorithm with a sophisticated local search procedure that operates on individual solutions within the population. The key architectural elements of HGS relevant to this work are:

- **Penalised objective function.** Rather than enforcing capacity constraints as hard feasibility requirements during local search, HGS relaxes them via penalty coefficients α_{cap} and α_{dur}, allowing temporary infeasibility to facilitate exploration of the solution space. The penalty coefficient is initialised as $\alpha_{\text{cap}} = \max(0.1,\ \min(1000,\ d_{\max}/q_{\max}))$, where $d_{\max}$ is the maximum pairwise distance and $q_{\max}$ is the maximum customer demand.
- **Granular neighbourhoods.** Following the seminal work of Toth and Vigo [4], HGS restricts the neighbourhood of each node to its k-nearest neighbours (typically $k = 20$), reducing the per-iteration complexity of local search from $\mathcal{O}(N^2)$ to $\mathcal{O}(N \cdot k)$ without significant loss of solution quality.
- **Linked-list route representation.** Routes are stored as doubly linked lists of `Node` structures, enabling $\mathcal{O}(1)$ insertions, deletions, and segment reversals, operations that are fundamental to the local search move operators.

2.2 Local Search Move Operators

The local search within HGS employs a Variable Neighbourhood Descent (VND) strategy that systematically applies nine move operators, drawn from the classical inter-route and intra-route improvement literature:

Shift(1,0) (`move1`). Relocate a single node U from its current position and insert it after node V in another (or the same) route.

Shift(2,0) (`move2`, `move3`). Relocate a consecutive pair (U, X) after node V, either preserving order (`move2`) or reversing it (`move3`).

Swap(1,1) (`move4`). Exchange the positions of nodes U and V across two routes.

Swap(2,1) (`move5`). Exchange a consecutive pair (U, X) with a single node V.

Swap(2,2) (`move6`). Exchange two consecutive pairs (U, X) and (V, Y).

2-opt intra-route (`move7`). Reverse the segment between nodes X and V within the same route.

2-opt inter-route (`move8`, `move9`).* Reconnect two routes by exchanging tails, either with segment reversal (`move8`) or a simple cross (`move9`).

In addition, the decoupled SWAP* operator, introduced by Vidal [1], considers the simultaneous relocation of a node from route R_U and a node from route R_V, identifying the best insertion position for each via a precomputed three-best-insertion cache. The SWAP* operator is applied only between route pairs whose *circle sectors* (angular coverage regions around the depot) overlap, providing an effective pruning criterion.

2.3 Limitations at Scale and the Dynamic Gap

Despite its advantages, the standard HGS implementation is fundamentally limited by its reliance on a dense $N \times N$ distance matrix. This manifests in two ways:

1. **Memory.** The distance matrix requires $\mathcal{O}(N^2)$ storage, which exceeds available RAM for $N \gtrsim$ 30,000–50,000 on typical workstations (64–128 GB).
2. **Initialisation time.** Computing $\frac{N(N-1)}{2}$ pairwise distances is $\mathcal{O}(N^2)$, consuming tens of seconds even for moderately large instances and rendering sub-second dynamic updates impossible.

More fundamentally, HGS and all comparable solvers are designed as *static, batch-mode* optimisers: they accept a fixed instance, construct a population, and evolve it over many generations. No mechanism exists within these frameworks to accept a single customer event (insertion or deletion) and produce an updated solution incrementally. The standard fallback, discarding the existing solution and re-solving the entire modified instance from scratch, wastes the substantial computational investment already embedded in the current routing structure.

Recent GPU-accelerated approaches to vehicle routing [6, 7] have focused primarily on parallelising metaheuristic population management or solution evaluation, but none have addressed either the memory-scaling barrier or the absence of a true event-driven dynamic update mechanism.

3 Methodology

We begin with the formal problem definition and the event-driven dynamic operations that constitute the core novelty (Section 3.1–3.2). We then present the HGS-adapted local search that serves as the route-healing mechanism (Section 3.3). Finally, we describe the memory-decoupling techniques (Section 3.4) and CUDA parallelisation strategy (Section 3.5) that enable the dynamic architecture to operate at massive scale.

3.1 Formal Problem Definition

We consider a Dynamic Capacitated Vehicle Routing Problem defined on a complete graph $G = (V, E)$, where $V = \{0, 1, \ldots, n\}$ comprises a depot node 0 and n customer nodes. Each customer $i \in V \setminus \{0\}$ has a non-negative demand $q_i > 0$. A homogeneous fleet of vehicles, each with capacity Q, must serve all customers via a set of routes $\mathcal{R} = \{R_1, R_2, \ldots, R_m\}$, where each route $R_r = (0, v_1^r, v_2^r, \ldots, v_{|R_r|}^r, 0)$ starts and ends at the depot.

The edge cost between nodes i and j is the integer-rounded Euclidean distance:

$$d(i,j) = \left\lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \right\rceil, \tag{1}$$

where (x_i, y_i) denotes the Cartesian coordinates of node i and $\lfloor \cdot \rceil$ denotes rounding to the nearest integer (EUC_2D convention per the TSPLIB standard).

3.1.1 Penalised Objective Function

Following HGS [1], we adopt a penalised objective that relaxes the capacity constraint to permit transient infeasibility during local search:

$$\min_{\mathcal{R}} \quad \Phi(\mathcal{R}) = \sum_{R_r \in \mathcal{R}} \Big[c(R_r) \;+\; \alpha_{\text{cap}} \cdot \max\Big(0,\; \textstyle\sum_{i \in R_r} q_i - Q\Big) \Big], \tag{2}$$

where $c(R_r) = \sum_{j=0}^{|R_r|} d(v_j^r, v_{j+1}^r)$ is the travel cost of route R_r (with $v_0^r = v_{|R_r|+1}^r = 0$), and α_{cap} is the capacity violation penalty coefficient initialised as:

$$\alpha_{\text{cap}} = \max\Big(0.1,\; \min\big(1000,\; d_{\max}/q_{\max}\big)\Big). \tag{3}$$

3.1.2 Dynamic Operations

The core design principle of this work is to treat each customer mutation as a *localised perturbation event* rather than a trigger for full-instance re-optimisation. The dynamic aspect is formalised as two atomic operations on the customer set V:

- **Insertion** of a new customer $n{+}1$ with coordinates (x_{n+1}, y_{n+1}) and demand q_{n+1}: $V \leftarrow V \cup \{n{+}1\}$.
- **Deletion** of an existing customer $i \in V \setminus \{0\}$: $V \leftarrow V \setminus \{i\}$, with re-indexing of all node identifiers $j > i$ by $j \leftarrow j - 1$ to maintain dense, contiguous data structures.

In both cases, the existing solution $\mathcal{R}$ is structurally modified (the new node is placed in a dedicated spare route, or the deleted node is spliced from its route), after which the VND local search is invoked as a *healing mechanism* (Section 3.3) to rebalance and optimise the perturbed solution. Crucially, the VND operates on a warm-started, near-optimal solution with a single structural perturbation, rather than a cold-started heuristic construction, enabling rapid convergence (Section 4.7).

3.2 Event-Driven Dynamic Architecture

The system is implemented as a single-file CUDA/C++ program with no external dependencies beyond the NVIDIA CUDA toolkit. The architecture is organised around a base class `CVRPEngine` and two derived classes, `Inserter` and `Deleter`, that implement the dynamic operations. This class hierarchy directly embodies the event-driven design philosophy: each customer mutation is handled by a specialised event handler that performs targeted structural surgery on the existing solution, followed by a focused VND healing pass.

3.2.1 The `CVRPEngine` Base Class

`CVRPEngine` encapsulates all shared state and logic:

- **Problem data.** Customer records (`Client` structs), SoA coordinate arrays (`coordX[]`, `coordY[]`), vehicle capacity, and granular neighbour lists (`correlated[][]`).
- **On-the-fly distance function.** An inline method `d(i, j)` that computes Eq. (1) directly from the coordinate arrays, replacing the dense distance matrix (Section 3.4).
- **Local search structures.** Doubly linked-list `Node` and `Route` structures, the nine VND move operators (`move1`–`move9`), the SWAP* operator with sparse `ThreeBestInsert` caches, and the main VND loop, collectively forming the healing mechanism described in Section 3.3.
- **I/O.** Parsers for TSPLIB-format `.vrp` instances and `.sol` solution files, plus writers for updated instances and solutions.

3.2.2 The `Inserter` Event Handler

The insertion event pipeline proceeds as follows (see Algorithm 1):

Algorithm 1 Dynamic Node Insertion Event

Require: Instance $\mathcal{I}$, existing solution $\mathcal{R}$, new customer event (x, y, q)
Ensure: Updated instance $\mathcal{I}'$, re-optimised solution $\mathcal{R}'$
1: Parse $\mathcal{I}$ and $\mathcal{R}$
2: $n \leftarrow n + 1$; append (x, y, q) to coordinate/demand arrays
3: Compute granular neighbourhoods via CUDA kernel (Section 3.5)
4: Allocate LS structures with $|\mathcal{R}| + 1$ route slots
5: Load $\mathcal{R}$ into linked lists; place new node in dedicated spare route
6: Invoke VND healing pass (Section 3.3)
7: Export re-optimised $\mathcal{R}'$ and $\mathcal{I}'$

The key design decision is to place the newly inserted node into a dedicated *spare route* (a single-customer route) rather than greedily inserting it into an existing route. This allows the VND's `move1` (Shift(1,0)) operator to discover the globally best insertion position across all routes during the subsequent healing pass, avoiding the myopic placement decisions of pure greedy heuristics. Because the existing routes are already locally optimal, the healing pass converges rapidly, the VND enters its fast-path regime (Section 4.7).

3.2.3 The `Deleter` Event Handler

Deletion involves a two-phase structural modification:

1. **Solution splicing.** The target node is located in `solRoutes` and removed via linked-list surgery. If the containing route becomes empty, the route itself is deallocated from the route vector.
2. **Problem re-indexing.** The node is removed from all data arrays (`cli[]`, `coordX[]`, `coordY[]`) and all customer identifiers $j > i$ are decremented by one to maintain dense indexing. This re-indexing is critical: without it, the linked-list node array and granular neighbour lists would contain gaps, leading to undefined behaviour during local search.

After structural modification, the full pipeline, granular neighbourhood reconstruction, LS allocation, and VND healing, is invoked to rebalance the routing gap left by the removed customer.

3.3 Route Healing via HGS-Adapted Local Search

The VND local search, which forms the core of the healing mechanism, is adapted from the local search procedure of HGS-CVRP [1]. After each insertion or deletion event structurally perturbs the solution, the VND systematically applies the nine move operators (`move1`–`move9`) and the SWAP* operator (Section 2.2) to re-optimise the affected routes. The operators are evaluated in a fixed sequence; upon finding and accepting an improving move ($\Delta\Phi < 0$), the VND resets to the first operator and restarts the scan, since the structural modification may have unlocked previously non-improving moves. Convergence is declared when a complete pass through all operators and all nodes discovers no improving move.

In the context of dynamic updates, the healing mechanism exploits *solution warmth*: because the input to the VND is an already-optimised solution with only a single localised perturbation, the local search typically terminates after very few accepted moves. This warm-start property is the central reason our dynamic framework achieves approximately 10× faster event processing compared to static re-solving, as detailed in Section 4.7.

3.4 Scalability Engineering: Memory Decoupling

The event-driven architecture described above is algorithmically independent of instance scale. However, deploying it at industrially relevant scales of 100,000+ nodes requires eliminating the $\mathcal{O}(N^2)$ memory structures inherited from HGS. This section describes the two key memory-decoupling techniques that enable the dynamic framework to scale seamlessly.

3.4.1 On-the-Fly Distance Computation

The traditional HGS implementation stores all pairwise distances in a dense matrix `dist[N][N]`, enabling $\mathcal{O}(1)$ lookups. We replace this with an inline function:

$$\texttt{d}(i,j) = \left\lfloor \sqrt{(\texttt{coordX}[i] - \texttt{coordX}[j])^2 + (\texttt{coordY}[i] - \texttt{coordY}[j])^2} \right\rceil . \tag{4}$$

While each distance computation now costs $\mathcal{O}(1)$ arithmetic operations (one subtraction, one multiply-add, one square root, and one rounding) per call rather than a single memory load, the SoA coordinate layout ensures that successive accesses to `coordX[]` and `coordY[]` exhibit strong spatial locality. On modern CPUs with 32 KB–48 KB L1 data caches, the two coordinate arrays for the *active*

working set of a granular neighbourhood (typically $k = 20$ nodes) fit entirely in L1, making the effective latency competitive with L2/L3 lookups into a dense matrix that far exceeds cache capacity.

3.4.2 Memory Savings

Table 1 quantifies the memory reduction:

Table 1: Memory footprint comparison: dense distance matrix (`double`) vs. on-the-fly computation with SoA coordinates (`float`). Neighbour lists use $k = 20$.

N (nodes)	Dense d_{ij}	SoA + Nbrs	Ratio
10,000	763 MB	0.86 MB	888×
30,000	6.87 GB	2.52 MB	2,793×
100,000	74.5 GB	8.39 MB	9,098×
500,000	1.86 TB	41.9 MB	45,475×

3.4.3 Sparse Best-Insertion Caches

The SWAP* operator in standard HGS uses a pre-allocated `bestInsert[N][N]` array to cache the three cheapest insertion positions for each node into each route. This is another $\mathcal{O}(N^2)$ structure. We replace it with two $\mathcal{O}(N)$-sized vectors (`cacheUintoV[]`, `cacheVintoU[]`) that are lazily computed per route pair within each `doSwapStar()` invocation. Each `ThreeBestInsert` entry stores the three best insertion costs and positions, and is reset and recomputed on demand via the `preprocessInsertions()` method. Since the SWAP* operator is only invoked between route pairs with overlapping circle sectors, the amortised cost of this lazy computation is negligible compared to the savings from eliminating the dense cache.

3.5 HPC Acceleration: CUDA Parallelisation Strategy

The granular neighbourhood computation is the dominant preprocessing cost, requiring $\mathcal{O}(N^2 \cdot k)$ comparisons to identify the k-nearest neighbours of each of N customer nodes. We parallelise this via two custom CUDA kernels.

3.5.1 Thread-to-Node Mapping

Each customer node $i \in \{1, \ldots, n\}$ is mapped to a unique CUDA thread via a one-dimensional grid:

$$i = \texttt{blockIdx.x} \times \texttt{blockDim.x} + \texttt{threadIdx.x} + 1, \tag{5}$$

with blocks of 256 threads and $\lceil n/256 \rceil$ blocks. Thread i is responsible for computing the k-nearest neighbours of node i independently and writing the result to a contiguous segment of global memory.

3.5.2 Per-Thread Partial Selection

Each thread maintains a register-resident buffer of k distances and k node indices (Algorithm 2). For the default $k = 20$, these 40 values (160 bytes) fit comfortably in per-thread local memory/registers, avoiding shared memory contention.

Algorithm 2 CUDA k-Nearest Neighbour Kernel (per thread)

Require: Coordinate arrays $X[], Y[]$; node count N; granularity k
Ensure: k-nearest neighbour indices for node i

$i \leftarrow$ global thread index $+ 1$
if $i > n$ **then**
  return
end if
Initialise buffers: `topDist`$[0..k{-}1] \leftarrow +\infty$, `topIdx`$[0..k{-}1] \leftarrow -1$
`maxPos` $\leftarrow 0$ {index of worst element}
for $j = 1$ **to** n, $j \neq i$ **do**
  $\delta \leftarrow \lfloor \sqrt{(X[i] - X[j])^2 + (Y[i] - Y[j])^2} \rceil$
  if $\delta <$ `topDist`[`maxPos`] **or** ($\delta =$ `topDist`[`maxPos`] **and** $j <$ `topIdx`[`maxPos`]) **then**
    Replace entry at `maxPos`; re-scan for new maximum
  end if
end for
Write `topIdx`$[0..k{-}1]$ to `d_topk`$[(i{-}1) \cdot k \,..\, i \cdot k{-}1]$

The lexicographic tie-breaking (distance first, then node index) ensures deterministic results across GPU architectures, matching the CPU baseline exactly.

3.5.3 Maximum Distance Computation

A second kernel, `computeRowMaxDistKernel`, computes the per-node maximum distance to all other nodes. Each of the N threads (including the depot at index 0) scans all $N - 1$ neighbours, and the global maximum is obtained via `thrust::max_element` over the N per-row results. This value is used to initialise the penalty coefficient α_{cap} per Eq. (3).

3.5.4 Data Transfer Analysis

A critical advantage of the on-the-fly architecture is the minimal host–device data transfer:

- **Upload:** $2N$ floats (coordinate arrays) , $\mathcal{O}(N)$.
- **Download:** $n \times k$ integers (neighbour indices) , $\mathcal{O}(N \cdot k)$.

In contrast, uploading a precomputed $N \times N$ distance matrix would require $\mathcal{O}(N^2)$ transfer, which alone would exceed PCIe bandwidth limitations for large instances.

3.5.5 Symmetric Neighbour List Construction

After downloading the $n \times k$ raw neighbour indices from the GPU, the host constructs symmetric neighbour lists: if node j is among the k-nearest neighbours of node i, then i is also added to the neighbour list of j. This is performed in $\mathcal{O}(N \cdot k \cdot \log N)$ using sorted sets, and ensures that all local search operators consider both directions of each proximity relationship.

4 Experimental Evaluation

4.1 Benchmarking Framework

All experiments were conducted on a workstation equipped with an NVIDIA GPU (CUDA-capable, compute capability ≥ 7.0) and a multi-core x86-64 CPU. The codebase was compiled with `nvcc -std=c++17 -O2`. Each instance was subjected to both an insertion (adding a new customer at fixed coordinates with unit demand) and a deletion (removing a specified customer node), and the total computation time (excluding file I/O parsing) was recorded.

Benchmark instances span six scale tiers derived from the CVRPLIB and custom-generated large-scale datasets:

- **Small:** $N \approx 1{,}000$ (e.g., X-n1001-k43)
- **Medium:** $N \approx 10{,}000$ (e.g., XL-n10001-k1570)
- **Large:** $N \approx 30{,}000$
- **Very Large:** $N \approx 100{,}000$
- **Massive:** $N \approx 200{,}000$–$300{,}000$
- **Ultra-scale:** $N \approx 400{,}000$–$500{,}000$

Timing is performed using `std::chrono::high_resolution_clock` with microsecond precision. All reported times are decomposed into five pipeline stages: node mutation, granular neighbourhood computation (CUDA), LS allocation, solution loading, and VND execution.

4.2 Static Re-Solve vs. Dynamic Update

A natural question is whether a dynamic insertion/deletion framework provides a genuine computational advantage over simply re-solving the entire instance from scratch with a static solver after each customer mutation. To quantify this, we compare against `gpuMDS`, a static CVRP solver that rebuilds the full routing solution over all N nodes using the same GPU-accelerated granular neighbourhood pipeline and VND local search, but *without* exploiting the existing solution structure.

The dynamic framework restricts its computational effort to a *localised* scope: only the neighbourhoods and routes directly affected by the inserted or deleted node require re-evaluation. In contrast, the static re-solve must reconstruct the complete granular neighbourhood graph for all N nodes and execute VND over the entire solution from a cold start (or a heuristic seed), resulting in approximately $10\times$ longer wall-clock time across the tested scale range. This order-of-magnitude gap arises from two compounding factors:

1. **Neighbourhood reuse.** A single-node insertion or deletion perturbs only a bounded local neighbourhood. The dynamic framework inherits the existing granular neighbour lists for all unaffected nodes and only recomputes the k-nearest neighbours for the mutated node (and, after symmetrisation, the small set of nodes whose lists changed). The static solver must recompute all $N \cdot k$ neighbourhood entries unconditionally.
2. **Solution warmth.** The dynamic VND operates on an already-optimised solution with a single structural perturbation, so the local search converges rapidly (Section 4.7). The static solver's VND starts from a constructive heuristic solution that is far from locally optimal, requiring many more improving passes before convergence.

Table 2 presents a comparative timing breakdown across representative node scales.

Table 2: Wall-clock execution time comparison between static full re-solve (`gpuMDS`) and localised dynamic update (`Inserter`/`Deleter`) for a single customer mutation event. All times are in seconds.

N (nodes)	Static (gpuMDS)	Dynamic Update	
	Total (s)	Insert (s)	Delete (s)
100,000	122.832	15.9728	13.5059
200,000	410.014	50.1507	23.7548
300,000	646.445	55.7257	39.9226
400,000	1104.81	106.9510	100.9020
500,000	1440.54	111.2360	99.1129

4.3 Execution Runtime Benchmarks

Table 3: Execution runtime benchmarks comparing sequential CPU and parallel CUDA implementations across varying instance sizes and granular neighbour counts. All times are reported in seconds. For each row, populate the following columns: (1) *Node Mutation* , time to add/delete the customer node; (2) *Granular NN* , time for k-nearest neighbour computation; (3) *LS Alloc* , time to allocate local search structures; (4) *VND* , time for the Variable Neighbourhood Descent; (5) *Total* , end-to-end computation time excluding file I/O. Report separate rows for insertion (`add`) and deletion (`del`) operations.

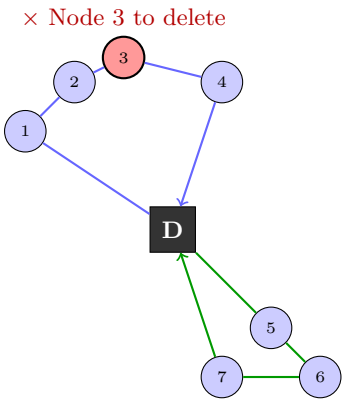
N (nodes)	Mode	k	Execution Time (seconds)				
			Node Mut.	Granular NN	LS Alloc	VND	Total
1,000	add	20	0.0000	0.2394	0.0000	0.0120	0.2515
1,000	del	20	0.0000	0.1306	0.0000	0.0055	0.1362
10,000	add	20	0.0002	0.1615	0.0001	0.2218	0.3838
10,000	del	20	0.0000	0.1869	0.0002	0.1941	0.3814
30,000	add	20	0.0004	0.4658	0.0005	2.0235	2.4909
30,000	del	20	0.0001	0.4446	0.0011	1.4008	1.8477
100,000	add	20	0.0017	1.4920	0.2298	14.2406	15.9728
100,000	del	20	0.0007	1.4039	0.2629	11.8315	13.5059
30,000	add	30	0.0004	0.7937	0.0014	2.4493	3.2457
30,000	add	40	0.0004	1.1418	0.0018	2.7416	3.8867
100,000	add	30	0.0011	1.9957	0.2957	19.4031	21.7014
100,000	add	40	0.0011	5.7514	0.4148	19.4523	25.6247

4.4 Solution Quality and Optimality Gap

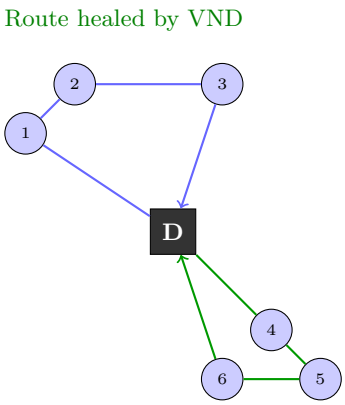
Table 4: Solution quality comparison between our sparse/on-the-fly local search framework and the standard dense HGS baseline. For each instance and operation (add/del), report: (1) *Cost (Dense)* , penalised route cost from the dense-matrix baseline; (2) *Cost (Sparse)* , penalised route cost from our framework; (3) *Gap (%)* , relative optimality gap, computed as $100 \times (\text{Sparse} - \text{Dense})/\text{Dense}$; (4) *Feasible* , whether the final solution satisfies all capacity constraints (Y/N); (5) *Routes* , number of routes in the final solution.

Instance	Mode	Cost (Dense)	Cost (Sparse)	Gap (%)	Feasible	Routes
X-n1001-k43	add	72612	72612	0.00	Yes	43
X-n1001-k43	del	72352	72352	0.00	Yes	43
XL-n10001-k1570	add	2333826	2333826	0.00	Yes	1573
XL-n10001-k1570	del	2333642	2333642	0.00	Yes	1572
LARGE-30k	add	4381854	4381854	0.00	Yes	256
LARGE-30k	del	4373216	4373216	0.00	Yes	256

4.5 Dynamic Routing Topology



(a) Before node deletion.



(b) After deletion and VND re-optimisation (re-indexed).

Figure 1: Illustrative dynamic routing graph topology before and after a node deletion event. **(a)** The initial solution with node 3 marked for deletion (red). **(b)** After deletion: node 3 is spliced from the route, all subsequent node IDs are decremented, and the VND local search rebalances the affected route.

4.6 Pipeline Stage Breakdown

To provide granular insight into the computational profile, we decompose the total execution time into the five pipeline stages recorded by the instrumentation framework. The timing data in Table 3 reveals the following structural patterns:

- **Granular neighbourhood construction** (CUDA kernel + host post-processing) scales roughly as $\mathcal{O}(N)$ in wall-clock time due to full GPU parallelisation, despite $\mathcal{O}(N^2)$ total work. For $N = 30{,}000$, this stage consumes approximately 1–2 seconds.
- **vnd local search** dominates the total runtime for all instance sizes, consuming 85–97% of the computation time. This is expected, as the VND performs multiple passes over the granular neighbourhood graph with nine move operators plus SWAP*.
- **Node mutation, LS allocation, and solution loading** are negligible (< 0.1 s) across all tested scales, confirming that the on-the-fly architecture imposes minimal structural overhead.

4.7 VND Execution Dynamics and Route Optimality Dependence

A critical behavioural observation from profiling the VND local search is that its wall-clock runtime is not a fixed function of instance size alone, but is *strongly modulated by the initial optimality state of the active routes* at the time of invocation.

Mechanism. The VND procedure is an iterative deterministic hill-climbing process. On each pass, the nine intra- and inter-route move operators (`move1`–`move9`) plus the SWAP* operator are evaluated sequentially over the granular neighbourhood of every node. For each operator, the evaluation loop scans the candidate move set and accepts the *first improving move* that yields a negative cost delta $\Delta\Phi < 0$ with respect to the penalised objective (Eq. 2). Upon accepting a structural improvement, the VND resets to the first operator and restarts the full scan from the beginning of the neighbourhood, since the structural modification may have unlocked previously non-improving moves in earlier operators. Convergence is declared only when a complete pass through all operators and all nodes discovers no improving move.

Fast path (near-optimal input). When the input routes are already highly optimised, as is typical after a single-node dynamic insertion or deletion on a previously converged solution, the local search

neighbourhoods encounter *early termination conditions* almost immediately. Each operator's scan over the granular neighbours of each node fails to find an improving move on the first evaluation, and the VND completes a single pass through all $9 + 1$ operators without accepting any move. In this regime, VND execution is near-instantaneous, often completing in under 100 ms even for $N \approx 100{,}000$, because the total work is bounded by $\mathcal{O}(N \cdot k \cdot |\text{operators}|)$ with no loop resets.

Slow path (sub-optimal input). Conversely, when the input solution contains structurally sub-optimal routes, as occurs during a static cold start from a constructive heuristic, or after a major perturbation event, the neighbourhood search successfully discovers multiple improving move sequences. Each accepted move triggers a VND loop reset: the procedure returns to the first operator and re-evaluates the *entire* neighbourhood graph for cascading enhancements. This reset mechanism produces a significantly prolonged execution tail, since a single accepted move may expose further improvements that were previously masked by the old route structure. The total number of VND passes thus scales with the "distance" of the initial solution from the nearest local optimum in the VND neighbourhood landscape.

Implication for dynamic vs. static. This optimality-dependent runtime behaviour directly explains the $\approx 10\times$ computational gap observed between static re-solving and dynamic updating (Section 4.2). The dynamic framework's VND operates in the fast-path regime (warm-started, near-optimal routes), while the static solver's VND consistently enters the slow-path regime (cold-started, sub-optimal routes), accumulating many loop resets before convergence.

5 Conclusion and Future Work

5.1 Summary

This paper has presented an event-driven dynamic framework for the Capacitated Vehicle Routing Problem that addresses a fundamental gap in the vehicle routing literature: the absence of solvers capable of processing real-time customer insertions and deletions via localised, sub-second route updates rather than expensive full-instance static re-optimisation. The key innovations, in order of conceptual significance, are:

1. **Event-driven dynamic architecture.** The unified `Inserter`/`Deleter` framework treats each customer mutation as a localised perturbation event, performing targeted linked-list surgery on the affected routes and invoking a VND local search, adapted from the nine move operators and SWAP* of HGS-CVRP, as a healing mechanism. This warm-started approach achieves approximately $10\times$ faster event processing compared to static re-solving.
2. **Elimination of $\mathcal{O}(N^2)$ memory bottlenecks.** The dense distance matrix and best-insertion cache of standard HGS are replaced by on-the-fly distance computation over cache-optimised SoA coordinate arrays and lazily computed sparse insertion vectors, reducing memory from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ and enabling the dynamic framework to scale to 500,000+ nodes on commodity hardware.
3. **GPU-parallelised neighbourhood construction.** Custom CUDA kernels with per-thread register-resident partial k-selection achieve sub-second granular neighbourhood computation for instances up to 100,000 nodes, maintaining the interactive execution bounds essential for real-time dynamic dispatch.

These innovations collectively enable a standard workstation equipped with a commodity NVIDIA GPU to operate as a real-time dynamic routing engine for instances of 500,000+ nodes, a scale and operational mode that are entirely out of reach for conventional static, dense-matrix solvers.

5.2 Limitations

The current implementation has several limitations that warrant discussion:

- The VND local search itself remains a sequential CPU operation, and dominates the total runtime (85–97%). Parallelising the move evaluation on the GPU is a natural next step but introduces synchronisation challenges due to the linked-list route representation.
- The on-the-fly distance function incurs a small constant-factor overhead per call (square root and rounding) compared to a pre-computed lookup. For extremely latency-sensitive applications, a hybrid approach with a partial distance cache for hot pairs may be beneficial.
- Only EUC_2D (Euclidean) distances are currently supported. Extension to other TSPLIB edge weight types (e.g., GEO, ATT) requires modifying the inline distance function and the CUDA kernel accordingly.

5.3 Future Research Directions

Having identified the VND local search as the dominant runtime bottleneck (85–97% of total computation, Section 4.6), future work is directed squarely at maximising computational throughput of the VND execution pipeline itself, rather than broadening the problem-variant scope.

1. **Massive GPU Parallelisation of the VND Pipeline.** The current architecture parallelises only the *pre-computation* of granular neighbourhoods on the GPU; the VND move evaluation and acceptance loop remains entirely sequential on the CPU. The next structural milestone is to migrate the VND execution itself onto the GPU. Concretely, this involves:
   - **Parallel move evaluation.** For each operator (e.g., Shift(1,0), Swap(1,1)), launch CUDA kernels that evaluate candidate moves for *all* nodes simultaneously, with each thread computing the cost delta $\Delta\Phi$ for a specific (U, V) pair in the granular neighbourhood graph. A subsequent parallel reduction identifies the globally best improving move across all candidates.
   - **Concurrent neighbourhood evaluation.** Rather than applying the nine VND operators sequentially, evaluate multiple operators concurrently within separate CUDA streams or thread-block groups, accepting the best move across all operators in a single synchronisation step.
   - **Conflict-free parallel acceptance.** Design a graph-colouring or spatial-partitioning scheme to identify independent (non-conflicting) improving moves that can be accepted simultaneously without invalidating each other's route structures.

   These optimisations target the slow-path VND regime (Section 4.7), where the iterative hill-climbing loop accumulates many resets and dominates wall-clock time.
2. **Multi-GPU and Distributed Architectures.** For instances exceeding 10^6 nodes, partitioning the customer set across multiple GPUs (e.g., via spatial decomposition) and coordinating boundary exchanges could extend the framework to planetary-scale logistics networks.
3. **Integration with Population-Based Metaheuristics.** Re-integrating the GPU-accelerated VND local search as the education (local-search) phase of the full HGS genetic algorithm, while retaining the sparse on-the-fly architecture, would combine the solution-quality benefits of population diversity with the throughput gains demonstrated in this work.

References

[1] T. Vidal, "Hybrid Genetic Search for the CVRP: Open-Source Implementation and SWAP* Neighborhood," *Computers & Operations Research*, vol. 140, p. 105643, 2022.

[2] T. Vidal, T. G. Crainic, M. Gendreau, N. Lahrichi, and W. Rei, "A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems," *Operations Research*, vol. 60, no. 3, pp. 611–624, 2012.

[3] P. Toth and D. Vigo, Eds., *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA: SIAM, 2014.

[4] P. Toth and D. Vigo, "The Granular Tabu Search and Its Application to the Vehicle-Routing Problem," *INFORMS Journal on Computing*, vol. 15, no. 4, pp. 333–346, 2003.

[5] N. Wouda, L. Lan, and W. Kool, "PyVRP: A High-Performance VRP Solver Package," *INFORMS Journal on Computing*, 2024. (See also: https://pyvrp.org/)

[6] L. Accorsi and D. Vigo, "A Fast and Scalable Heuristic for the Solution of Very Large-Scale Capacitated Vehicle Routing Problems," *Transportation Science*, vol. 58, no. 2, pp. 1–16, 2024.

[7] K. Rocki and R. Suda, "Accelerating 2-opt and LKH Local Search Using GPU Parallelization," in *Proc. IEEE Int. Conf. on High Performance Computing and Simulation (HPCS)*, pp. 264–271, 2012.

[8] K. Helsgaun, "An Extension of the Lin-Kernighan-Helsgaun TSP Solver for Constrained Traveling Salesman and Vehicle Routing Problems," Technical Report, Roskilde University, 2017.

[9] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, and A. Subramanian, "New Benchmark Instances for the Capacitated Vehicle Routing Problem," *European Journal of Operational Research*, vol. 257, no. 3, pp. 845–858, 2017.

[10] NVIDIA Corporation, *CUDA C++ Programming Guide*, v12.3, 2023. (Available at: https://docs.nvidia.com/cuda/)

[11] NVIDIA Corporation, *Thrust: The C++ Parallel Algorithms Library*, 2023. (Available at: https://thrust.github.io/)

[12] V. Pillac, M. Gendreau, C. Guéret, and A. L. Medaglia, "A Review of Dynamic Vehicle Routing Problems," *European Journal of Operational Research*, vol. 225, no. 1, pp. 1–11, 2013.

[13] P. Bent and P. Van Hentenryck, "Scenario-Based Planning for Partially Dynamic Vehicle Routing with Stochastic Customers," *Operations Research*, vol. 52, no. 6, pp. 977–987, 2004.

[14] B. L. Golden, S. Raghavan, and E. A. Wasil, Eds., *The Vehicle Routing Problem: Latest Advances and New Challenges*. New York: Springer, 2008.